

PVL 4

Parsa Besharat - 69365

(a) Optimization Techniques Applied

To optimize the code, I focused on removing the "roadblocks" that typically slow down a processor's pipeline. I applied loop unrolling and striding by incrementing the loop counter by 2 instead of checking a modulo every iteration. This completely removes the expensive division-based math and the conditional branches that usually force the CPU to guess which way the code will flow. I also used strength reduction by replacing the slow $\text{pow}(x, 2)$ function with a simple $x \times x$ multiplication, which is much easier for the hardware to process. Finally, I applied algebraic simplification to avoid the costly square root function. Instead of calculating a root and comparing it to 10.0, the code now compares the squared sum directly against 100.0, only performing the heavy math when it is strictly necessary.

Optimized Code:

```
void process_fast(double* a, double* b, double* processed, int N) {
    for (int i = 0; i < N; i++) {
        processed[i] = 0.0;
    }

    for (int i = 0; i < N; i += 2) {
        double v_sq = (a[i] * a[i]) + (b[i] * b[i]);
        if (v_sq > 100.0) {
            processed[i] = sqrt(v_sq);
        }
    }
}
```

Also I would like to tell that another method which could be quite useful is via using the OMP SIMD to apply SIMD vectorization method. Although we could also manually apply the code explicitly so that it manages the vector registers (AVX/SSE).

However this technique has its own problems. For example, in order to do so, we must Stop Striding, meaning that we have to loop to iterate one by one (i++).

This code here is being optimized via SIMD:

```
void process_simd(double* a, double* b, double* processed, int N) {
    #pragma omp simd
    for (int i = 0; i < N; i++) {
```

```

    if (i % 2 == 0) {
        double v_sq = a[i]*a[i] + b[i]*b[i];
        if (v_sq > 100.0) {
            processed[i] = sqrt(v_sq);
        } else {
            processed[i] = 0.0;
        }
    } else {
        processed[i] = 0.0;
    }
}
}

```

(b) Additional Serial Optimization Techniques

1. **Shrinking the working set.** This involves reducing the amount of memory an algorithm accesses at any given time so that the data fits entirely within the CPU's L1, L2, or L3 cache. Since fetching data from the main RAM is significantly slower than pulling it from the onboard cache, keeping the "active" data within arm's reach prevents the processor from idling while it waits for memory operations to complete.

2. **Function inlining.** Every time a program calls a function, there is a small amount of overhead involved in jumping to a new memory address and setting up a local workspace on the stack. For small functions that are called millions of times inside a loop, this overhead adds up quickly. By inlining, the compiler essentially copies the function's logic directly into the spot where it was called, removing the "commute" entirely and allowing the code to run in a single, uninterrupted flow.